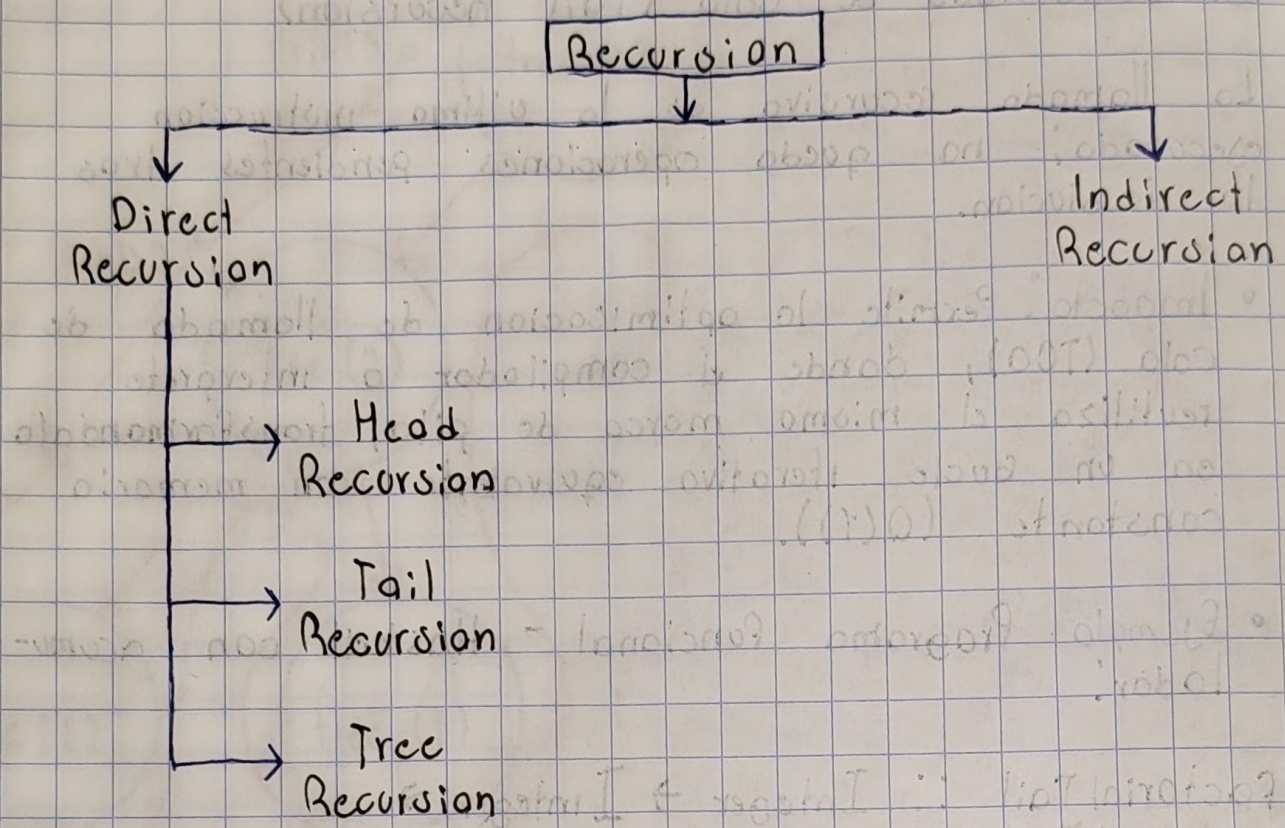
TIPOS DE RECURSIVIDAD

La recursividad es el mecanismo fundamental de control y flujo en la programacion funcional y logica, ya que va remploza a las bucles imperativas (for, while). Dependiendo de como y donde se realiza la llamada recursiva, se clasifica en distintas tipos.



1. Tipos de Recursividad Segun el Flujo de Llamadas

A. Recursividad Lineal / Sin cola (Non-Tail Recursion)

La llamada recursiva no es lo ultima operacion. Tras regresar de la llamada, la funcion debe realizar calculos pendientes.

- Impacto: Consume espacio en la pila de llamadas (O(n) porque debe recordar las operaciones pendientes.

- Ejemplo (Programacion funcional - Haskell):

```
factorial :: Integer > Integer
factorial 0 = 1
factorial n = n * factorial (n - 1) -- Debe esperar el resultado
```

B. Recursividad de Cola (Tail Recursion)

La llamada recursiva es la ultima instruccion ejecutada; no queda operaciones pendientes tras la devolucion.

- Impacto: Permite la optimizacion de llamada de cola (TCO), donde el compilador o interprete reutiliza el mismo marco de pila, transformandola en un bucle iterativo equivalente con memoria constante (O(1)).

- Ejemplo Programa funcional - Haskell con acumulador:

```
factorialTail :: Integer → Integer →
factorialTail 0 acc = acc
factorialTail n acc = factorialTail (n - 1) (n * acc)
```

C. Recursividad Multiple / en Arbol

Ocurre cuando una funcion realiza dos o mas llamadas recursivas dentro de un mismo cuerpo de ejecución.

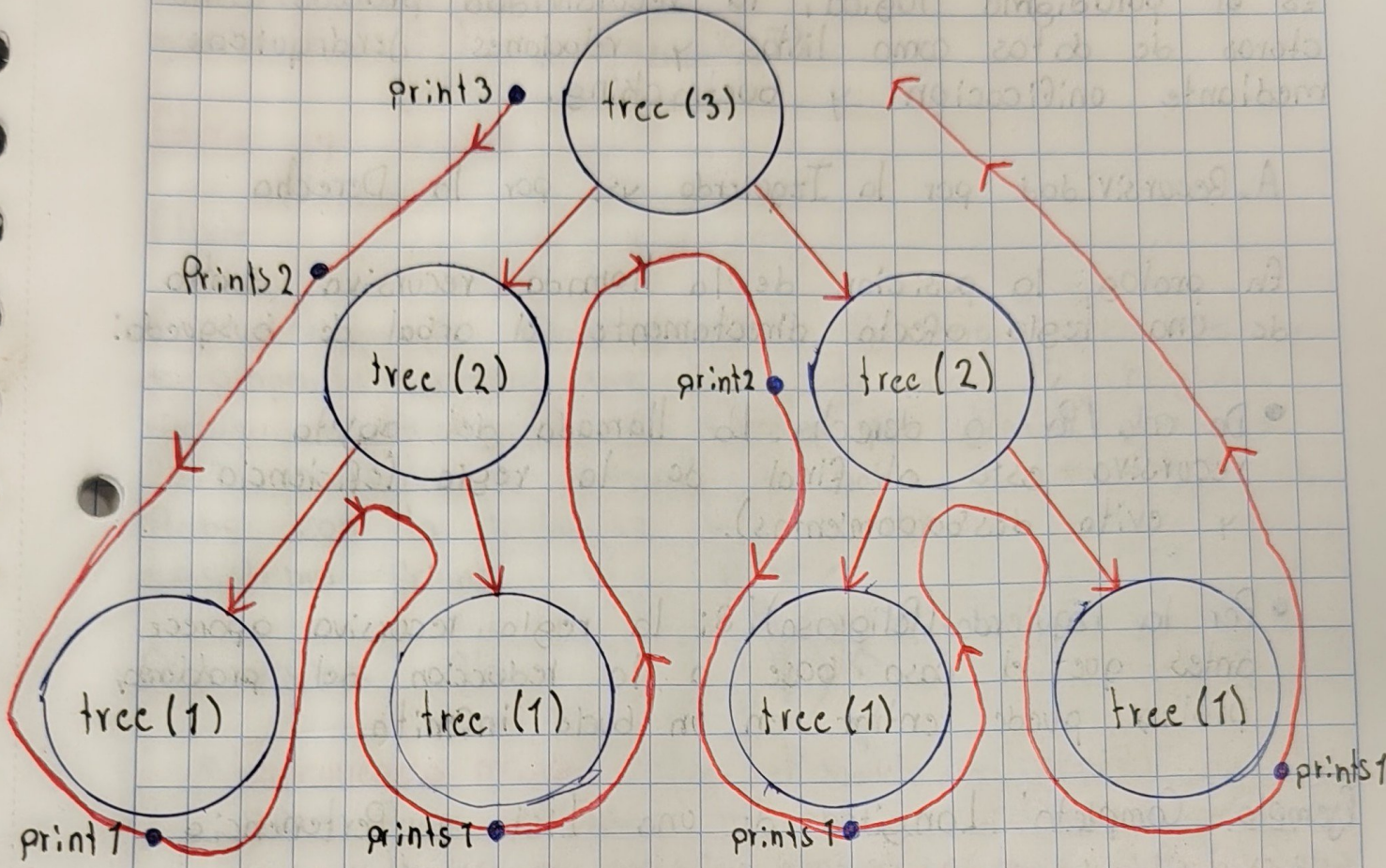
- Ejemplo (Programacion funcional - Haskell):

```
fibonacci :: Integer → Integer
```

Norma

fibonacci 0 = 0
fibonacci 1 = 1
fibonacci n = fibonacci (n-1) + fibonacci (n-2)

Tree Recursion



D. Recursividad Mutua / Cruzada (Indirect Recursion)

Dos o mas funciones se invocan de forma ciclica entre si: (A llama a B, y B llama a A).

- Ejemplo (Programacion Logica / Prolog):

```
%. Verificar si un numero natural es par o impar de forma cruza
es_par(0).
es_par(N) :- N > 0, N1 is N-1, es_impar(N1).
```

es_impar(1).
es_impar(N) :- N > 1, N1 is N-1, es_par(N1).

2. Recursividad en la Programacion Logica (Prolog)

Es el paradigma logica, la recursividad procesa estructuras de datas como listas y relaciones jerárquicas mediante unificacion y backtracking.

A. Recursividad por la Izquierdo vs. por la Derecha

En prolog, la posicion de la llamada recursiva dentro de una regla afecta directamente el arbol de busqueda:

- De cola / Por la derecha: La llamada del objeto recursivo esta al final de la regla (eficiencia y evita desbordamientos).
- Por la izquierda (Peligroso): Si la regla recursiva aparece antes que el caso base o la reduccion del problema, prolog puede entrar en un bucle infinito.

Ejemplo Completo: Longitud de una Lista y Pertenencia

```
%--- Ejemplo 1: Recursividad de cola para calcular la longitud---
% caso base: la longitud de una lista vacia con acumulador es el acum
longitud([], Acc, Acc).

% Caso recursivo: procesamos la cabeza [H], pasamos a la cola [T] e incrementamos el acumulador.
longitud([_|T], Acc, Lenght) :-
    NuevaAcc is Acc + 1,
    longitud(T, NuevaAcc, Lenght). % Llamada al final de la regla
```

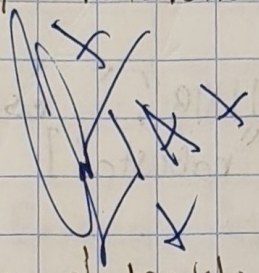
```
% Predicado principal:
% ?- longitud([a, b, c], 0, L).
% L = 3.

%--- Ejemplo 2: Recorrido de estructuras (Pertenencia a una lista) ---
miembro(X, [X|_]).
miembro(X, [_|T]) :- miembro(X, T).
```

Elixir y Erlang

Elixir

Es un lenguaje de programacion funcional, concurrente y de proposito general que se ejecuta sobre la maquina virtual de Erlang (BEAM). Creado por José Valim en 2012, combina la sintaxis amigable y productiva de Ruby con la robustez y tolerancia a fallos del ecosistema Erlang.



Caracteristicas Clave

- **Concurrencia masiva:** Utilizo el modelo de actores de la BEAM. Puede gestionar millones de procesos ligeros e independientes en paralelo consumiendo muy pocos recursos.
- **Tolerancia a fallas:** Basado en la filosofia "let it crash" (dejo que falle). Los procesos son supervisados individualmente y se reinician automaticamente si ocurre un error, sin afectar al resto de la aplicacion.
- **Programacion funcional:** Los datos son inmutables y no existen efectos secundarios ocultos, lo que facilita el razonamiento sobre el codigo y las pruebas automatizadas.

Norma

- **Interoperabilidad total:** Es 100% compatible con Erlang y sus bibliotecas existentes sin costo de rendimiento.

Ejemplo básico de código

Elixir utiliza el operador pipe (|>) para encadenar funciones de manera clara y legible:

```
defmodule Demo do
  def procesar_textos(lista_strings) do
    lista_strings
    |> Enum.map(&String.trim/1)
    |> Enum.map(&String.downcase/1)
    |> Enum.filter(fn texto -> String.length(texto) > 3 end)
  end
end

Demo.procesar_textos(["ELIXIR", " es ", " Robusto "])
# Resultado: ["elixir", "robusto"]
```

Ecosistema Principal

Herramienta	Uso Principal
Phoenix	Framework web de alto rendimiento para aplicaciones en tiempo real (WebSockets, LiveView).
Mix	Herramienta de compilación integrada para gestionar proyectos, tareas y dependencias.
Ecto	Capa de abstracción de base de datos y validación de datos.
Nerves.	Plataforma para desarrollo de sistemas embebidos e Internet de las cosas (IoT).

Norma

Cuando Eligir Elixir? Es ideal para sistemas distribuidos, servicios de mensajeria o chat, plataformas de streaming, arquitecturas de microservicios y aplicaciones web donde la bajo latencia y la alta disponibilidad sean indispensables

ERLANG

Es un lenguaje de programacion funcional, concurrente y distribuida diseñado para construir sistemas masivamente escalables, con alta disponibilidad y tolerancia a fallas.

Desarrollado originalmente por Joe Armstrong, Robert Virding y Mike Williams en Ericsson a finales de la decada de 1980, fue creada para gestionar la infraestructura de redes telefonicas que no podian permitirse ni un segundo de caido.

Caracteristicas principales:

- Modelo de actores: En Erlang, todo se organiza en torno a procesos ligeras aislados entre si que comunican informacion enviando y recibiendo mensajes de forma asincrona.
- Tolerancia a fallos: Diseñado con la filosofia de que las errores ocurririan. En lugar de llenar el codigo de excepciones defensivas, los procesos caen de forma aislada y los Supervisores las reinician automaticamente a un estado estable.
- Actualizacion de Codigo en Caliente (Hot Code Swapping): Permite modificar o actualizar el codigo de una aplicacion en produccion sin detener ni

interrumpir a los usuarios.

- **Programación Funcional Inmutable:** Los datos no cambian una vez creados, lo que elimina las condiciones de carrera (race conditions) y facilita el comportamiento concurrente.

- **Máquina Virtual BEAM:** El motor donde corre Erlang (y lenguajes modernos como Elixir), optimizado para baja latencia, manejo eficiente de memoria y gestión de millones de procesos concurrentes

Ejemplo Básico de Código

Ejemplo sencillo de un módulo en Erlang que implementa un contador a proceso que recibe mensajes:

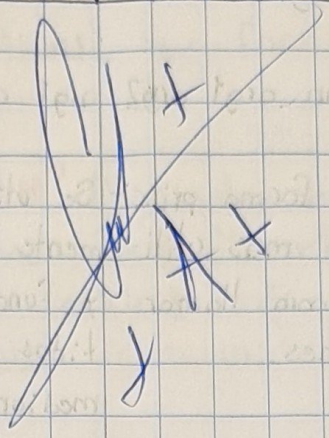
```
-module(servidor_mensajes).
-export([bucle/1]).

bucle(Estado) ->
  receive
    {hola, Nombre} ->
      io:format("Hola, ~p!~n", [Nombre]),
      bucle(Estado + 1);
    {total} ->
      io:format("mensajes procesados: ~p~n", [Estado]),
      bucle(Estado);
    stop ->
      io:format("Deteniendo servidor...~n", [])
  end
```

Norma

	F. Prefijas	F. Infijas	F. Postfijas
Definicion	La función se escribe antes de sus argumentos.	La función o operador se coloca entre dos operandos.	La función se escribe después de los operandos.
Sintaxis	funcion arg1 arg2	arg1 operador arg2	arg1 arg2 operador
Uso	Es la forma principal y más utilizada para llamar funciones.	Se utiliza principalmente con operadores y funciones convertidos a infijas mediante bodikis(1).	No existe soporte nativo para la notación postfija
Cantidad de Operandos	Uno o más argumentos.	Generalmente dos operandos.	Generalmente dos operandos.
Legibilidad	Ideal para funciones complejas o con varios parámetros.	Muy intuitiva para operaciones matemáticas y operaciones.	Puede ser menos intuitivo para quienes no conocen RPN.
Ventajas	Uniformidad, claridad y facilidad de composición funcional.	Expresiones más naturales y fáciles de leer.	Reduce el uso de paréntesis en algunos lenguajes.
Desventajas	Las expresiones matematicas pueden verse más extensa.	Suele limitarse a operaciones binarias.	No está disponible directamente en Haskell.
Aplicacion Principal	Llamadas a funciones definidas por el usuario o del lenguaje	Operaciones aritméticas, logicas y camparaciones.	Utilizada en calculadoras y algunos lenguajes especializadas

Ejemplo	max 5 8	5 `max` 8	5 8 max
Resultado	8	13	Error en Haskell



INTERVALOS

Los intervalos se gestionan principalmente a traves de rangos y enumeraciones (a traves de rangos) de listas utilizando la sintaxis de puntos suspensivos (..), los cuales estan respaldadas por el sistema de tipos y la potente evaluacion perezosa (lazy evaluation) del lenguaje.

OPERADORES

En Haskell, los operadores son esencialmente funciones que se escriben en notacion infixa (entre sus dedos argumentos). El lenguaje incluye una amplia variedad de operadores integradas para operaciones matematicas, comparaciones, logica, manipulacion de listas y composicion de funciones.

EVALUACION PEREZOSA

La evaluacion perezosa (lazy evaluation o call-by-

Norma

need) es uno de los pilares fundamentales y mas ~~definitivos~~ destintivos de Haskell. A diferencia de lenguajes imperativos u otros lenguajes funcionales estrictos, Haskell no evalua una expresion hasta que su valor es estrictamente necesario para continuar con el programa.